Markov Chains: Python Implementation

Markov Chain Object-Oriented Implementation

Below is the Python implementation of a Markov Chain, encapsulating core functionalities such as n-step transitions, calculating stationary distributions, identifying absorbing states, and determining communicating classes using the Floyd-Warshall algorithm.

```python
import numpy as np

class MarkovChain:
    def __init__(self, states, transition_matrix):
        # Initializing our Markov Chain.

        self.states = states
        self.transition_matrix = np.array(transition_matrix)

        # Validate the matrix by its 2 basic rules
        assert self.transition_matrix.shape[0] == self.transition_matrix.shape[1], "Matrix must be a square."
        for row in self.transition_matrix:
            assert np.isclose(sum(row), 1.0), f"Row probabilities must sum to 1. Found: {sum(row)}"

    def get_n_step_transition(self, n):
        # To Calculate the n-step transition matrix using matrix exponentiation.
        return np.linalg.matrix_power(self.transition_matrix, n)

    def calculate_stationary_distribution(self):
        # To Calculate the stationary distribution (pi) where pi*P = pi.
        # This is equivalent to finding the left eigenvector associated with eigenvalue 1.

        # First Transpose the matrix to find left eigenvectors using standard right eigenvector functions
        eigenvalues, eigenvectors = np.linalg.eig(self.transition_matrix.T)
        # Then find the index of the eigenvalue that is closest to 1
        eigen_1_index = np.argmin(np.abs(eigenvalues - 1.0))
        # Then extract the corresponding eigenvector
        stationary_vector = eigenvectors[:, eigen_1_index].real
        # And finally normalize the vector so probabilities sum to 1
        stationary_distribution = stationary_vector / np.sum(stationary_vector)

        return stationary_distribution

    def display_stationary(self):
        # To print the stationary distribution in a readable format
        pi = self.calculate_stationary_distribution()
        print("\n>>> Stationary Distribution <<<")
        for state, prob in zip(self.states, pi):
```

```
            print(f"State {state}: {prob:.4f} ({prob*100:.2f}%)")

    def get_absorbing_states(self):
        # To get absorbing states
        absorbing = []
        for i, state in enumerate(self.states):
            if np.isclose(self.transition_matrix[i, i], 1.0):
                absorbing.append(state)
        return absorbing

    def simulate_path(self, start_state, num_steps):
        # To simulates a random walk through the Markov Chain.

        # The starting state should be a valid state ofc
        if start_state not in self.states:
            raise ValueError(f"State '{start_state}' not found in state space.")

        current_state_idx = self.states.index(start_state)
        path = [start_state]

        for i in range(num_steps):
            next_state_idx = np.random.choice(
                len(self.states),
                p=self.transition_matrix[current_state_idx]
            )
            path.append(self.states[next_state_idx])
            current_state_idx = next_state_idx

        return path

    def get_communicating_classes(self):
        # To identify communicating classes (Strongly Connected Components) using reachability to help classify states as transient or recurrent.

        n = len(self.states)
        # First we will need a Boolean reachability matrix (which answers, can i reach j directly?)
        reachability = np.zeros((n, n), dtype=bool)
        np.fill_diagonal(reachability, True)
        reachability = reachability | (self.transition_matrix > 0)

        # Using Floyd-Warshall algorithm
        for k in range(n):
            for i in range(n):
                for j in range(n):
                    reachability[i, j] = reachability[i, j] or (reachability[i, k] and reachability[k, j])

        # Group states into communicating classes
        visited = set()
        classes = []
        for i in range(n):
            if i not in visited:
                # A class includes all states 'j' where 'i' can reach 'j' AND 'j' can reach 'i'
```

```
                current_class_indices = [j for j in range(n) if
    reachability[i, j] and reachability[j, i]]
                classes.append([self.states[idx] for idx in
    current_class_indices])
                visited.update(current_class_indices)

        return classes


if __name__ == "__main__":
    # EXAMPLE 1: Weather Model

    # Defining the states
    weather_states = ['Sunny', 'Cloudy', 'Rainy']

    # Defining the Transition Matrix (P)
    # Row 1 (Sunny): 60% chance tomorrow is Sunny, 30% Cloudy, 10%
    Rainy
    # Row 2 (Cloudy): 40% Sunny, 40% Cloudy, 20% Rainy
    # Row 3 (Rainy): 20% Sunny, 50% Cloudy, 30% Rainy
    P = [[0.6, 0.3, 0.1],
        [0.4, 0.4, 0.2],
        [0.2, 0.5, 0.3]]

    # Initializing the chain
    mc = MarkovChain(weather_states, P)

    # Q. What happens after 5 days?
    print("\n>>> 5-Step Transition Matrix (P^5) <<<<")
    print(np.round(mc.get_n_step_transition(5), 4))

    # Q. Calculate the Stationary Distribution
    mc.display_stationary()

    # Q. Simulate 10 days of weather starting from a Sunny day
    print("\n>>> 10-Day Weather Simulation <<<")
    weather_path = mc.simulate_path('Sunny', 10)
    print(" -> ".join(weather_path))


    # EXAMPLE 2: The Gambler's Ruin (Absorbing States & Classes)
    print("Example 2: Gambler's Ruin ($0 to $3)")

    gambler_states = ['$0', '$1', '$2', '$3']
    # p = 0.5 (fair coin flip to win or lose $1)
    gambler_P = [
        [1.0, 0.0, 0.0, 0.0],
        [0.5, 0.0, 0.5, 0.0],
        [0.0, 0.0, 0.0, 1.0],
        [0.5, 0.0, 0.5, 0.0]
    ]

    gambler_mc = MarkovChain(gambler_states, gambler_P)

    print("\n>>> Absorbing States <<<")
    print(gambler_mc.get_absorbing_states())

    print("\n>>> Communicating Classes <<<")
```

```
    classes = gambler_mc.get_communicating_classes()
    for i, c in enumerate(classes):
        # A class with only 1 state that is absorbing is recurrent.
        # A class that can reach an absorbing state but isn't absorbing itself is transient.
        print(f"Class {i+1}: {c}")

    print("\n>>> Gambler Simulation (Starting at $1) <<<")
    gambler_path = gambler_mc.simulate_path('$1', 7)
    print(" -> ".join(gambler_path))
```